

Computer Programming Summer Interest Group

Session 19 / Concurrency 2

Isaiah 40:27-31

Why do you say, O Jacob, and speak, O Israel, “My way is hidden from the Lord, and my right is disregarded by my God”?

Have you not known? Have you not heard? The Lord is the everlasting God, the Creator of the ends of the earth. He does not faint or grow weary; his understanding is unsearchable.

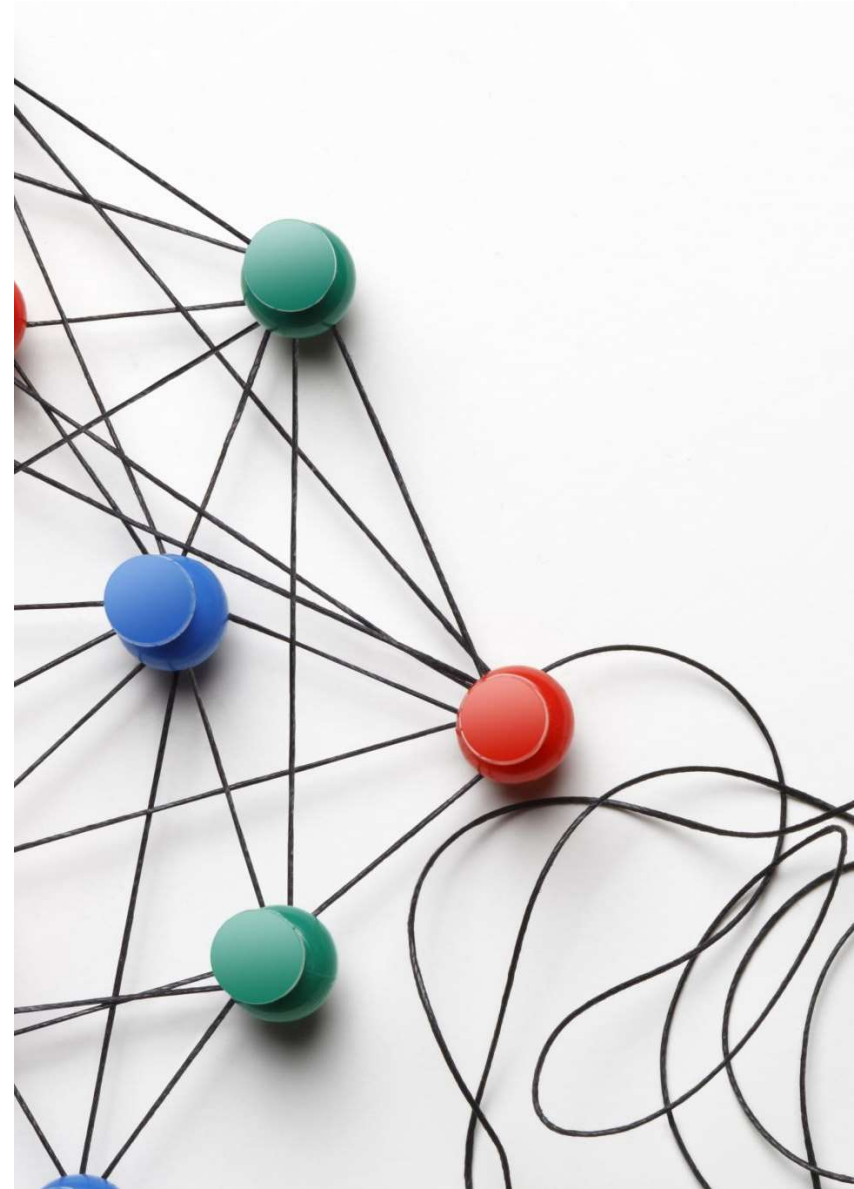
He gives power to the faint, and to him who has no might he increases strength.

Even youths shall faint and be weary, and young men shall fall exhausted;

but they who wait for the Lord shall renew their strength; they shall mount up with wings like eagles; they shall run and not be weary; they shall walk and not faint.

Objectives

- By the end of this session, you will be able to:
 - Explain the concept of asynchronous programming and how it differs from threading.
 - Use `async` and `await` keywords to define and run coroutines.
 - Manage concurrent coroutines using `asyncio` tasks.
 - Use `asyncio` to handle I/O-bound tasks efficiently.



Concurrency with the Python asyncio package

- This is the second of three sessions on concurrency
- In the previous session we looked at using threads to achieve concurrency
- This session covers asynchronous input and output (asyncio)
- The forms of input and output (I/O) we have looked at until now have been “blocking”
 - The thread does not continue processing work while the I/O operations are being performed
- The asyncio allows non-blocking operations
 - After operations are started subsequent processing can continue without waiting for the operation to complete
 - Useful in many I/O operations
 - Not limited to I/O operations

Common use cases for asyncio

Database queries

Reading from or writing to disk

Waiting for a network request (i.e. download a document or web page)

Reading from or writing to data peripheral devices

High level description of asyncio

Functions can be defined as “coroutines”

Coroutines are not executed immediately when called, but a reference to an object that can be referenced later.

This reference will be evaluated later, possibly concurrently with other coroutines.

Coroutines all run in the same thread and require less overhead, and start more quickly than Python threads

Basic asyncio Example

- Import the **asyncio** package to use its features
- All **asyncio** coroutines are defined as functions with the **async** prefix
- All coroutines must be run inside an event loop
 - **asyncio.run** is used to create an event loop for **asyncio**
 - It must call a coroutine
 - This is the only way to call a coroutine from a regular python function.
- The **await** keyword is used inside a coroutine to invoke other coroutines
- In the example, each invocation of **say_after** is invoked in sequence
- This is no different than when using standard Python sequential code
 - The wait time for running this is the sum of the individual wait times for each invocation of **say_after**.

```
import asyncio

async def say_after(
    delay: float, what: str) :
    await asyncio.sleep(delay)
    print(what)

async def main() -> None:
    await say_after(1, "hello")
    await say_after(2, "world")

if __name__ == "__main__":
    asyncio.run(main())
```

Using asyncio.create_task

- This sample creates tasks that are executed concurrently
- Define each task using the **asyncio.create_task** function
- Here each of the tasks are run concurrently
- In this case main ends only when all tasks complete
- Total wait time is the largest of the individual task wait times.

```
import asyncio

async def say_after(delay: float, what: str):
    await asyncio.sleep(delay)
    print(what)

async def main() -> None:
    task1 = asyncio.create_task(
        say_after(1, "hello"))
    task2 = asyncio.create_task(
        say_after(2, "world"))

    await task1
    await task2

if __name__ == "__main__":
    asyncio.run(main())
```


Using asyncio.gather

- This performs similarly to using `create_tasks` but in a more compact notation
- All of the coroutines passed to `asyncio.gather` are run concurrently
- The `await asyncio.gather` function ends only when all of the functions passed to it have all completed.

```
import asyncio

async def say_after(delay: float, what: str):
    await asyncio.sleep(delay)
    print(what)

async def main() -> None:
    await asyncio.gather(
        say_after(1, "hello"),
        say_after(2, "world"))

if __name__ == "__main__":
    asyncio.run(main())
```



Activities

- Rework the thread pool activity from the previous session using asyncio instead of threads.
- Run a block of up to ***n*** functions at a time.
- Once done take the next block of ***n*** functions.